A Fault-Tolerant Distributed Voting System with Leader Election and Consistent Vote Replication

Ganesh Deshmukh, Adwait Kamble, Advait Nathe, Nikhil Puppalwar
Department of Computer Engineering (Regional Language), PCET's Pimpri Chinchwad College of Engineering, Pune, India

***Abstract*—**Voting systems are widely used in colleges, organizations, communities, and online platforms to collect opinions and determine collective decisions. Conventional voting applications commonly rely on a centralized server for receiving, validating, storing, and aggregating votes. Although centralized systems are simple to implement, dependence on a single server can result in service unavailability when that server fails and can complicate recovery of the latest voting state. This paper presents the design of a Fault-Tolerant Distributed Voting System using leader election, in which several server nodes cooperate to provide the voting service. One node acts as the current coordinator while the remaining nodes maintain distributed state and participate in coordination. The proposed system employs heartbeat-based failure detection to identify coordinator failure and uses either the Bully Algorithm or the Ring Algorithm to elect a replacement coordinator [1], [2]. Lamport logical clocks are used to establish a logical ordering of distributed events [3]. Voting information is replicated across participating nodes, and each vote is assigned a unique Vote ID to reduce the possibility of duplicate processing during normal operation and recovery. The inter-node communication layer is implemented using gRPC, which provides remote procedure calls between distributed services [12]. Docker containers are used to run multiple server nodes on a single host for controlled experimentation [13]. The system is intended as a small-scale experimental platform for studying the interaction of leader election, failure detection, logical clocks, replication, and fault recovery. Experiments are planned using clusters of 3, 5, and 7 nodes. Election time, message overhead, vote processing time, recovery time, throughput, lost votes, and duplicate votes will be measured to compare the Bully and Ring algorithms.

Keywords **—**Distributed Systems, Distributed Voting, Leader Election, Bully Algorithm, Ring Algorithm, Lamport Logical Clock, Fault Tolerance, Replication, gRPC, Docker.

I. INTRODUCTION

Distributed systems consist of multiple computing nodes that communicate and cooperate to perform a common task. When a system depends on a single coordinating node, failure of that node can interrupt the service. Leader-election mechanisms provide a way for distributed processes to reorganize after failure and select a coordinator among the active processes [1].

Voting applications provide a useful example for demonstrating this problem. In a simple centralized voting application, clients send their votes to a single server. The server validates the requests, stores the votes, and produces the final tally. Such an architecture is straightforward but creates a dependency on a single processing and storage point.

A distributed voting architecture can instead use multiple cooperating nodes. However, distributing the nodes introduces additional coordination requirements. The system must determine which node is currently responsible for coordination, detect when that node becomes unavailable, select a replacement, and preserve the latest voting state.

Leader election is a fundamental problem in distributed computing. Garcia-Molina studied the reorganization of active processes after failures and the election of a coordinator to continue system operation [1]. Chang and Roberts subsequently investigated decentralized extrema-finding in ring configurations, establishing an important foundation for ring-based election algorithms [2].

Another issue is event ordering. In a distributed environment, different nodes can receive messages at different times, and physical clocks cannot by themselves provide a complete ordering of distributed events. Lamport

introduced logical clocks to establish a relation between distributed events and provide a mechanism for ordering them [3].

Consensus protocols such as Raft further demonstrate the relationship between leadership and replicated state. Ongaro and Ousterhout designed Raft around leader election, log replication, and safety [4]. More recent work such as SCRaft and RaftOptima investigates ways to reduce election or communication overhead in Raft-based systems [5], [6].

The proposed project focuses on a more classical and educational distributed-system design. Instead of implementing a complete consensus protocol, it combines Bully and Ring leader election, heartbeat-based failure detection, Lamport logical clocks, replicated voting state, unique Vote IDs, gRPC communication, and Docker-based deployment in a single experimental application.

II. BACKGROUND AND MOTIVATION

A. Centralized Voting Architecture

In a conventional centralized system, all voting operations are directed to one server. The server performs vote validation, storage, and aggregation. This architecture has several limitations:

1) Single Point of Failure

Failure of the central server can make the voting service unavailable. Leader election research specifically addresses the need to reorganize active processes after failures [1].

2) Vote-State Loss

When voting state exists only on one server, a failure can make the latest state difficult or impossible to recover.

3) Processing Bottleneck

All requests are directed to the same server, concentrating processing and communication load.

4) Manual Recovery

A basic centralized deployment generally requires a server restart or administrative recovery procedure after a failure.

These limitations motivate the use of multiple cooperating nodes.

B. Motivation for Distributed Voting

The proposed system attempts to improve continuity of service by maintaining several cooperating server nodes. The system uses:

Leader Election — selects a new coordinator after failure.

Heartbeat Monitoring — detects coordinator unavailability.

Lamport Clocks — provides logical event ordering [3].

Replication — preserves voting state across nodes.

Unique Vote IDs — helps prevent duplicate vote processing.

gRPC — provides structured communication between distributed services [12].

The combination of these mechanisms forms the central research focus of the project.

III. PROBLEM STATEMENT

To design a distributed voting system that automatically detects coordinator failure, elects a replacement using the Bully or Ring algorithm, maintains a logical order of voting events using Lamport clocks, and recovers the latest replicated voting state without losing or double-counting votes.

IV. OBJECTIVES

The objectives of this research are:

1. To study leader-election and fault-tolerance techniques used in distributed systems, and to implement the Bully and Ring leader-election algorithms together with heartbeat-based coordinator failure detection in a multi-node voting environment.
2. To use Lamport logical clocks for ordering distributed voting events, and to replicate voting information across participating nodes with unique Vote IDs to reduce duplicate processing.
3. To implement inter-node communication using gRPC and to deploy multiple nodes using Docker containers.
4. To experimentally compare the Bully and Ring algorithms using clusters of 3, 5, and 7 nodes.
5. To measure election time, message overhead, vote processing time, recovery time, throughput, lost votes, and duplicate votes as part of this comparison.V. LITERATURE REVIEW

Recent Distributed Systems research on leader election, consensus, fault tolerance, replicated logs, and distributed decision-making relates directly to the proposed system, because a distributed poll needs its nodes to coordinate, recover from failures, and keep a consistent view of the votes.

Election research covers both the classical Bully and Ring algorithms and newer versions that try to cut message overhead and election time. Consensus research, especially on Raft [4], is relevant because Raft combines leader election with replicated log management, the same two ideas this system relies on. Work on distributed voting itself studies how decisions are made when voters are split across groups, which gives a theoretical view of polling across nodes.

Review of Specific Studies

The studies below were chosen because they relate directly to the main parts of the proposed system: leader election, fault tolerance, consensus, replicated state, and distributed voting.

A. Leader Election

Patel and Tere [7] propose an election algorithm that exchanges fewer messages than Bully and Ring while recovering better from faults. Related work on message-efficient Bully variants reduces message traffic by making one node responsible for running the election and assigning each node a time span. Studies of election on rings where message contents can be corrupted show that an O(n·ID_max) message cost is close to the best possible in that model. Other work elects a coordinator that tolerates node failures and security threats instead of simply picking the highest ID.

B. Raft-based Consensus

Kondru and Saranya [6] add proxy leaders to Raft in RaftOptima and report lower latency with up to 25 servers. Related work makes Raft Byzantine fault tolerant using secret sharing and a committee that shares the leader's load, and tests Raft under node failures and network partitions with a discrete-event simulator. Wang et al. [5] speed up elections in SCRaft by naming a successor leader in advance, and related work lets followers call an election when the leader performs poorly.

C. Distributed Voting

Filos-Ratsikas and Voudouris [9], [10] analyse how well district-based voting mechanisms choose a winner. Dang and Hwang [11] survey consensus methods across distributed systems, blockchain, and voting, tracing the idea from classical voting theory to Paxos and Raft.

Table I summarises these studies. Together they show that leader election, fault tolerance, replicated state, and consensus are closely linked problems. Raft-based work treats election and log replication as the core of consistent distributed state [4]–[6], while other work continues to refine Bully, Ring, and fault-tolerant coordinator election [1], [2], [7], [8].

TABLE I
Summary of Reviewed Literature on Leader Election, Consensus, and Fault Tolerance

Sr. No.	Paper Title (Source, Year)	Architecture / Algorithm	Major Findings	Limitations	Relevance to Distributed Systems
1	Enhancing Election Algorithms for Distributed Systems: Reducing Message Complexity and Improving Fault Tolerance (IJARCS, 2025)	Enhanced Bully and Ring with optimized message passing	Reduces message complexity by 30–40% vs. classical Bully; improves fault tolerance under multiple node crashes	Simulation-based; no container or real-network deployment	Directly supports comparison of Bully vs. Ring and fault-tolerant coordinator election
2	SCRaft: A Consensus Algorithm with Pre-Designated Successive Leader for Reduced Election Latency (IEEE HPCC, 2024)	SCRaft (Raft variant with pre-designated successor leader)	Reduces election latency by 50–70% vs. standard Raft; avoids vote splitting during leader failure	Focused on Raft consensus; does not evaluate classical Bully/Ring algorithms	Useful for discussing how leader election can be optimized beyond classical algorithms; baseline for election-latency metrics
3	Deterministic Self-Stabilising Leader Election for Programmable Matter with Constant Memory (arXiv Preprint, 2024)	Self-stabilizing leader election with constant memory per node	Guarantees unique leader election even after arbitrary transient faults; uses O(1) memory per process	Targets programmable matter (nano-scale); not applicable to server clusters	Relevant for fault tolerance and recovery after coordinator crashes; shows self-stabilization concepts
4	PureLottery: Fair and Bias-Resistant Leader Election (arXiv Preprint, 2024)	PureLottery protocol (cryptographic leader election)	Achieves fairness (each node has equal probability of becoming leader) and bias-resistance even with malicious nodes	Requires cryptographic primitives; higher computational overhead	Useful for discussing fairness properties in leader selection; contrasts with highest-ID-wins Bully algorithm
5	FTI-TMR: A Fault Tolerance and Isolation Algorithm for Distributed Systems (arXiv Preprint, 2025)	FTI-TMR (Fault Tolerance and Isolation with Triple Modular Redundancy)	Uses max-heap and majority voting for leader election; tolerates up to f faulty nodes in a 3f+1 system	Focused on TMR architectures; not evaluated for voting applications	Directly relevant to coordinator election and fault detection module; shows integration of voting with leader selection
6	Near-Linear Time Leader Election in Multiagent Networks (AAMAS, 2025)	Randomized leader election with O(n log n) message complexity	Achieves near-linear time leader election in large-scale multiagent networks	Assumes synchronous communication; not evaluated for crash faults	Useful for discussing election latency metrics and message complexity trade-offs

Sr. No.	Paper Title (Source, Year)	Architecture / Algorithm	Major Findings	Limitations	Relevance to Distributed Systems
7	A Novel Raft Consensus Algorithm Combining Comprehensive Evaluation and Byzantine Fault Tolerance (CB-Raft) (Electronics MDPI, 2025)	CB-Raft (Raft with reputation-based leader election and Byzantine fault tolerance)	Improves consensus performance by 25–35%; resists Byzantine nodes through reputation scoring	Focused on Byzantine faults; higher overhead than crash-fault models	Useful for contrasting crash-fault (this system's model) vs. Byzantine-fault models; shows reputation-based leader selection
8	GFEA: Leader Election Algorithm for Choosing a Group Decision Facilitator (SET Journal, 2024)	GFEA (hybrid Bully + Paxos for ad hoc networks)	Combines Bully algorithm with Paxos consensus for fault-tolerant leader election; integrates failure detection	Focused on wireless ad hoc networks; not evaluated for containerized systems	Directly relevant to integration of election with failure detection; shows hybrid algorithm design
9	Exact Fault-Tolerant Consensus (Univ. of Glasgow Tech Report, 2023)	Exact consensus with voting and decision phases	Proposes exact fault-tolerant consensus protocol with formal guarantees on safety and liveness	Theoretical focus; no implementation or performance evaluation	Useful for discussing vote aggregation and consistency in the final tally; formal background on consensus
10	Leader Election in a Distributed CAN-Based Multi-Microcontroller System (IEEE ICECCME, 2023)	Self-stabilizing and criterion-based leader election; uses leader, assistant and co-assistant structure.	Provides dynamic leader election based on node performance and availability. The approach aims to reduce election messages and improve system availability and fault tolerance.	Designed for CAN-based multi-microcontroller systems; not evaluated in a server-based or containerized voting environment.	Relevant to leader election, fault tolerance, coordinator selection and failure recovery. It supports the leader-election component of the proposed voting system.

VI. RELATED WORK

A. Leader Election

Garcia-Molina presented the problem of electing a coordinator after distributed-system failures and developed election mechanisms for reorganizing active nodes [1]. This work provides the classical basis for coordinator election in the proposed system.

Chang and Roberts proposed an improved decentralized extrema-finding algorithm for processes arranged in a circular configuration [2]. The work demonstrates how processes in a ring can collectively determine a unique process without relying on a central controller.

More recent research continues to investigate improvements to leader-election mechanisms. Patel and Tere studied election algorithms with an emphasis on message complexity and fault tolerance [7]. Mohammedi Taieb and Laredj proposed GFEA, a leader-election approach for group decision-support environments that incorporates multiple selection criteria and recovery mechanisms [8].

These studies demonstrate that leader election remains an important mechanism for maintaining coordination in distributed applications.

B. Logical Clocks

Lamport introduced logical clocks to address the ordering of events in distributed systems [3]. The work establishes the concept that distributed events can be related using a logical ordering even when a globally synchronized physical clock is unavailable.

The proposed system applies this principle to voting operations. Each node maintains a logical clock and attaches logical timestamps to relevant distributed operations.

C. Consensus and Replicated State

Raft is a consensus algorithm designed around leader election, replicated logs, and safety [4]. Its structure illustrates how leadership and replicated state can be integrated into a distributed system.

Wang et al. proposed SCRaft, a Raft-related approach targeting faster and more stable leader transitions [5]. The work highlights the importance of election latency and service continuity in leader-based distributed systems.

Kondru and Saranya introduced RaftOptima, which modifies Raft by using proxy leaders for some replication-related tasks and evaluates its effect on scalability and latency [6].

Although these approaches are related to the proposed project, the present work deliberately focuses on classical Bully and Ring election rather than implementing a complete Raft consensus protocol. The objective is to provide a smaller experimental system in which individual Distributed Systems concepts can be studied and measured.

D. Distributed Voting

Filos-Ratsikas and Voudouris studied distributed voting in which voters are partitioned into districts and local voting results contribute to the final outcome [9]. Their work demonstrates that distributed voting introduces its own decision-making considerations.

The same authors later revisited distributed voting and studied deterministic and randomized mechanisms under different information assumptions [10].

Dang and Hwang provided a survey of consensus methods spanning distributed systems, blockchain, and voting [11]. Their work demonstrates the connection between voting, consensus, and distributed decision-making.

The proposed project addresses a different aspect of distributed voting: maintaining service availability and voting state when distributed server nodes experience crash failures.

VII. RESEARCH GAP

Existing research addresses leader election, consensus, fault tolerance, replicated state, and distributed voting from different perspectives.

Classical leader-election research provides mechanisms for selecting a coordinator after failures [1], [2]. Raft-based research combines leadership with replicated state management [4]–[6]. Distributed-voting research investigates voting mechanisms and decision quality in distributed voting environments [9], [10].

However, these research directions are generally concerned with individual algorithmic or theoretical problems rather than providing a small application-oriented implementation that combines several fundamental distributed-system mechanisms in one voting platform.

The proposed project addresses this implementation-oriented gap by integrating:

- Bully and Ring leader election,
- heartbeat failure detection,
- Lamport logical clocks,
- replicated voting state,
- unique Vote IDs,
- automatic coordinator recovery,
- gRPC communication, and
- Docker-based multi-node deployment.

The resulting platform provides a controlled environment for experimentally studying the relationship between leader election, communication overhead, recovery, and voting-state preservation.

VIII. PROPOSED SYSTEM

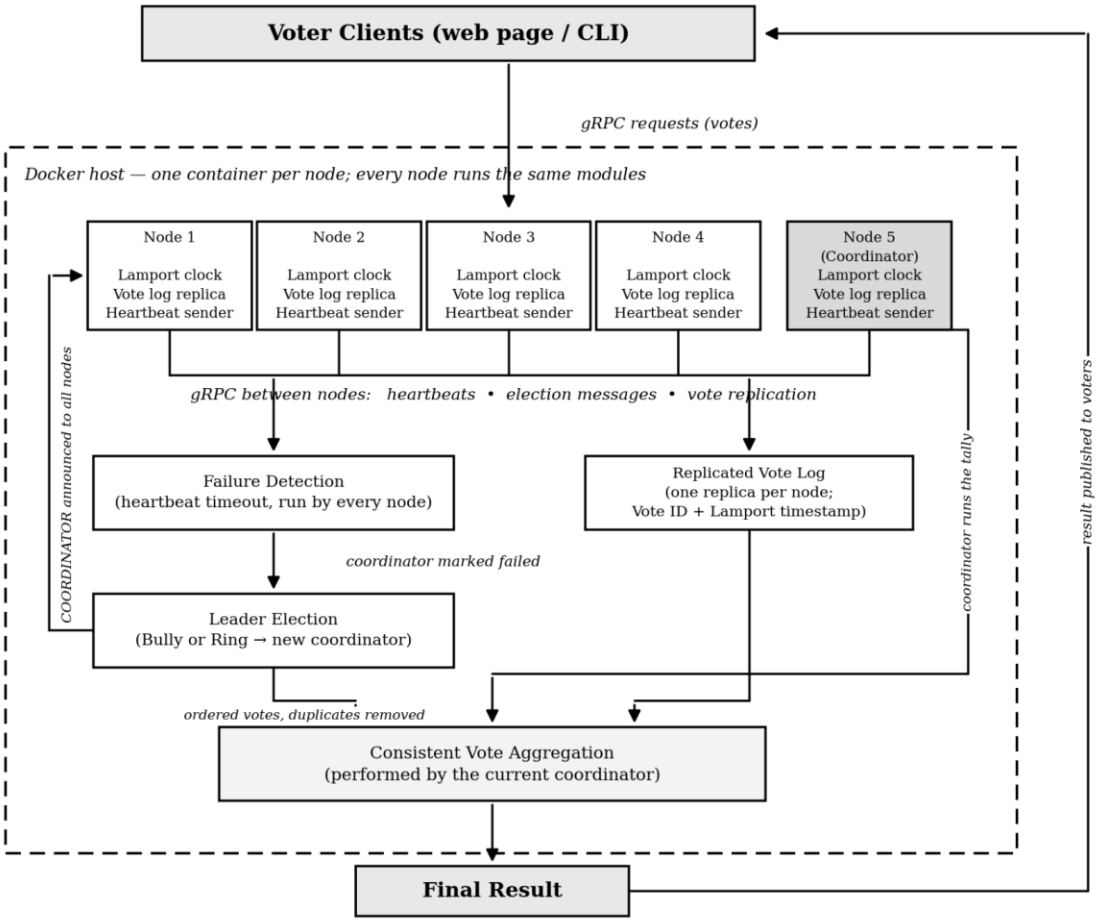


Fig. 1. Proposed architecture of the Distributed Voting System with Leader Election.

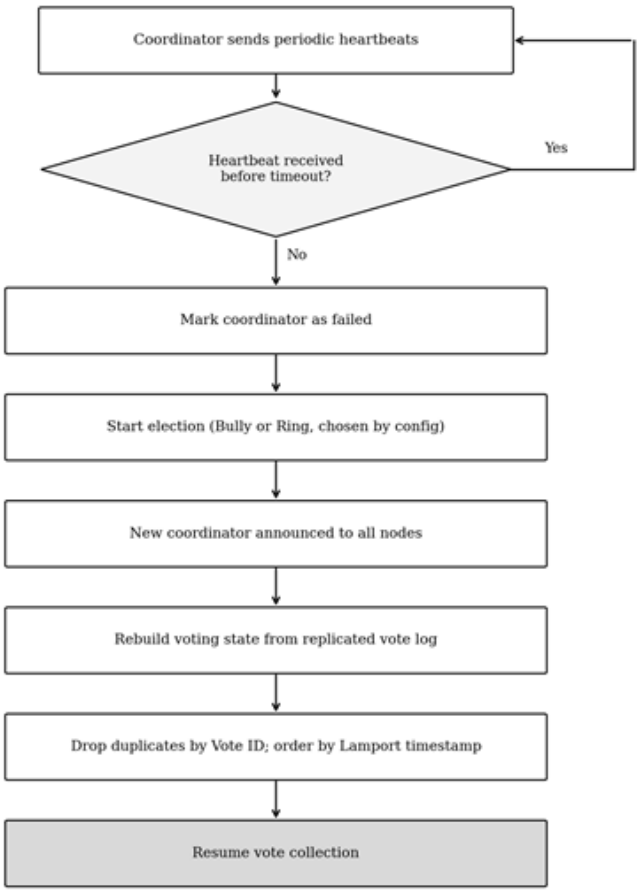


Fig. 2. Coordinator failure detection and recovery flow.

The system will be tested by changing the number of nodes and deliberately stopping the current coordinator. Election time, message overhead, vote processing time, recovery time, throughput, duplicate votes, and lost votes will be measured. This gives an experimental comparison of Bully and Ring while demonstrating practical concepts from the Distributed Systems syllabus.

REFERENCES

[1] H. Garcia-Molina, "Elections in a distributed computing system," IEEE Transactions on Computers, vol. C-31, no. 1, pp. 48–59, Jan. 1982, doi: 10.1109/TC.1982.1675885.

[2] E. J. H. Chang and R. Roberts, "An improved algorithm for decentralized extrema-finding in circular configurations of processes," Communications of the ACM, vol. 22, no. 5, pp. 281–283, May 1979, doi: 10.1145/359104.359108.

[3] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Communications of the ACM, vol. 21, no. 7, pp. 558–565, Jul. 1978, doi: 10.1145/359545.359563.

[4] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proc. USENIX Annual Technical Conference (USENIX ATC 14), 2014, pp. 305–319.

[5] Y. Wang, H. Luo, G. Sun, and H. Yu, "SCRaft: Achieving fast and stable elections in Raft consensus algorithm," in Proc. IEEE Int. Conf. High Performance Computing and Communications (HPCC), 2024, pp. 561–568, doi: 10.1109/HPCC64274.2024.00081.

[6] K. K. Kondru and R. Saranya, “RaftOptima: An optimised Raft with enhanced fault tolerance, and increased scalability with low latency,” IEEE Access, vol. 12, pp. 105974–105989, 2024, doi: 10.1109/ACCESS.2024.3435978.

[7] S. Patel and G. Tere, “Enhancing election algorithms for distributed systems: Reducing message complexity and improving fault tolerance,” International Journal of Soft Computing and Engineering, vol. 15, no. 1, pp. 16–22, Mar. 2025, doi: 10.35940/IJSCE.F8203.15010325.

[8] S. Mohammedi Taieb and M. A. Laredj, “GFEA: Leader election algorithm for choosing a group decision support system facilitator,” Science, Engineering and Technology, vol. 5, no. 2, pp. 53–74, May 2025, doi: 10.54327/set2025/v5.i2.216.

[9] A. Filos-Ratsikas, E. Micha, and A. A. Voudouris, “The distortion of distributed voting,” Artificial Intelligence, vol. 286, Art. no. 103343, 2020, doi: 10.1016/j.artint.2020.103343.

[10] A. Filos-Ratsikas and A. A. Voudouris, “Revisiting the distortion of distributed voting,” Theory of Computing Systems, vol. 68, pp. 1138–1159, 2024, doi: 10.1007/s00224-024-10171-1.

[11] D. T. Dang and D. Hwang, “Consensus-based methods for distributed systems, blockchain, and voting: A survey,” Journal of Information and Telecommunication, vol. 9, no. 2, pp. 237–260, 2025, doi: 10.1080/24751839.2024.2416728.

[12] gRPC Authors, “Introduction to gRPC,” gRPC Documentation, 2024. [Online]. Available: https://grpc.io/docs/what-is-grpc/introduction/

[13] Docker Inc., “What is Docker?” Docker Documentation, 2026. [Online]. Available: https://docs.docker.com/get-started/docker-overview/

[14] X. Deng, Z. Yu, W. Xiong, K. Li, and H. Liu, “A novel Raft consensus algorithm combining comprehensive evaluation partitioning and Byzantine fault tolerance,” The Journal of Supercomputing, vol. 80, no. 18, pp. 26363–26393, 2024, doi: 10.1007/s11227-024-06438-6.